

Yahboom DOFBOT 6 軸アームの AIPL 実装

— Capability 推論・TinyML 視覚認識・強化学習による サイクル最適化と、その検証 —

2026 年 7 月 15 日

Abstract

Yahboom DOFBOT (6DOF / Raspberry Pi 5) を対象に、AIPL でライン作業（コンベアの部品を手首カメラで認識し、色の一致する棚へ格納する）を記述・実行し、その周辺を整備した記録である。主張は次の 3 点に集約される。第一に、AIPL の Capability（効果）推論は本物の処理系上で機能し、実際に筆者の実装バグを検出した。効果は最小権限を型で強制し、PlannerActor は駆動も永続化もできないことが保証される。第二に、手首カメラ画像を分類する TinyML（純 Python・2,355 パラメータ）を実レンダ画像から学習し、その出力のみで棚を決定するようにした。ただし課題自体が易しく、テスト精度は 100% である。第三に、Cross-Entropy Method によりサイクル時間を 6.300 秒から 5.164 秒へ 18.0% 短縮した。最も重要な発見は副産物であり、手調整のベースラインはそもそも実機で成立していなかった——サーボへ上限の 1.93 倍の角速度を要求していた。強化学習は該当フェーズを遅くすることで実機で成立させ、他を詰めて全体を縮めている。なお本報告の結果はすべてシミュレーション上のものであり、物理の DOFBOT は接続していない。

1 対象システムと本報告の位置づけ

対象は 3D シミュレータ `aipl_line_simulator`（`localhost:8022`）と、それを駆動する AIPL ソース `aipl/dofbot_xinu.abcl` である。作業内容は次の繰り返しである。

1. コンベアで流れてきた部品（赤・立方体／青・円柱／緑・六角柱）がストッパで停止する。
2. アームが部品の真上（撮像姿勢）へ移動する。
3. 手首カメラで撮像し、TinyML で色を認識する。
4. 認識した色に対応する棚の空きスロットを確保する。
5. 逆運動学で解いた姿勢で把持し、旋回して棚へ置く。棚が満杯（4 個）なら出荷して空ける。

順序が「見てから決める」ことに注意されたい。手首カメラは工具軸方向を見下ろしているため、アームが部品の上へ行くまで部品を見られない。撮像前に分類する構成では、カメラは原理的に何も寄与しない。

報告に含めない前提。 物理の DOFBOT はネットワーク上に存在せず、本報告の数値はすべてシミュレータおよび AIPL 処理系上の実測である。実機接続時の経路は用意してあるが（§7）、実機で走らせた結果ではない。

2 アーキテクチャ — 12 Xinu と役割分離

6 軸すなわち 6 個のシリアルバスサーボそれぞれに `soft-Xinu` を 2 台割り当て、計 12 Xinu とする。

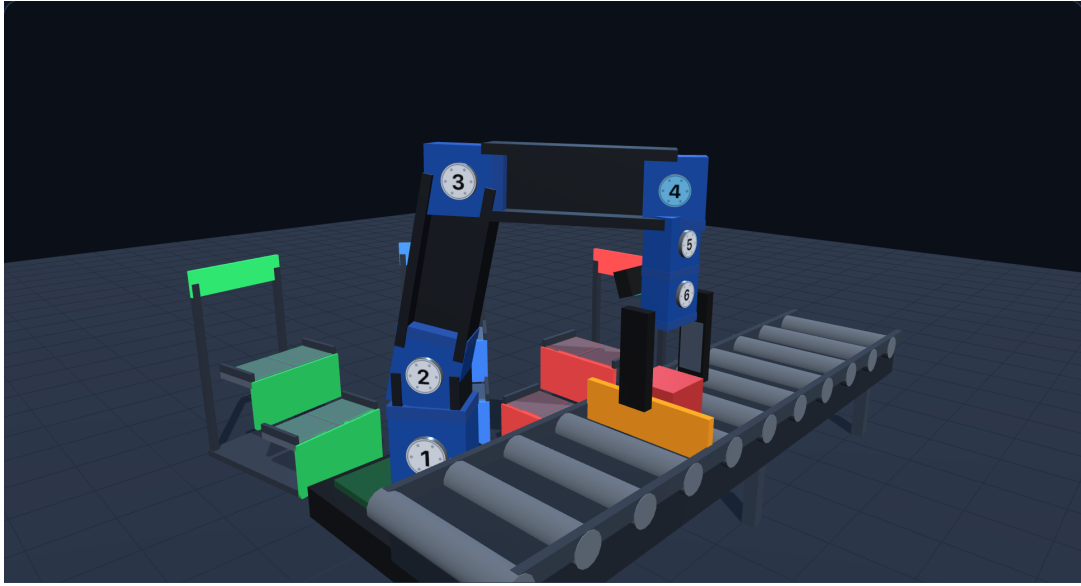


Figure 1: 作業セル。アーム中央、左が緑の棚、手前がコンベア。銀色ホーンの数字がサーボ ID（ID4 が点灯＝駆動中）。棚は色ごとに 3 台、雛壇 2 段 × 2 スロット。

アクター	効果	役割
ServoMotor×6	!{mut, net}	各軸 Xinu-A。実機サーボを駆動できる唯一の役割
ServoSensor×6	!{mut, net}	各軸 Xinu-B。角度計測のみ（read-only）
Kinematics	!{net}	逆運動学。純計算
ArmMover	!{mut, net}	ID1-ID5 を目標姿勢へ。駆動は各モータへ委譲
ConveyorActor	!{mut, net}	ベルト駆動・ストップ
CameraActor	!{net}	手首カメラ撮像
RecognitionActor	!{ai, mut, net}	TinyML 推論。ai を持つ唯一の役割
ShelfActor	!{mut, net}	棚のスロット割当・出荷
SafetyActor	!{fs, mut, net}	fs を持つ唯一の役割。サイクルの永続記録
PlannerActor	!{ai, net}	Pick&Place の計画。駆動も永続化もできない
Coordinator	!{ai, fs, mut, net}	12 Xinu の生成と巡回

最小権限がここに現れる。作業全体を指揮する PlannerActor が !{ai, net} しか持たないことは、計画アクターが暴走してもサーボを直接動かさないことを意味する。駆動できるのは ServoMotor と ConveyorActor だけであり、永続化できるのは SafetyActor だけである。これは規約ではなく型で強制される。

図1 に CG を示す。銀色ホーンの数字がサーボ ID であり、Arm_serial_servo_write(id, angle, ms) の id および画面右のアクターカードと一対一に対応する。駆動中のサーボは番号が点灯する（図では ID4）。

3 Capability（効果）推論

3.1 仕組み

AIPL の効果は *gradual* である。処理系は各メソッド本体が呼ぶ一次作用（primitive）から効果集合を観測し、ソースに申告がある場合に限り「申告 ⊇ 観測」を検査する。本作業で DOFBOT 実機デバイスのプリミティブを処理系へ追加し、次の効果を与えた。

プリミティブ	効果	根拠
Arm_serial_servo_write(id,angle,ms)	!{mut}	実機サーボを物理的に動かす
Arm_serial_servo_read(id)	(なし)	計測のみ。何も動かさない
dofbot_belt_drive(cmd)	!{mut}	コンベアもアクチュエータ
dofbot_camera_grab()	(なし)	センサ読取
tinymml_load(path)	!{fs}	重みをファイルから読む
tinymml_infer(model,features)	!{ai}	net を持たない

tinymml_infer が !{ai} のみで !{net} を伴わない点は設計上の主張である。既存の ai_call 系（LLM 呼び出し）は !{ai, net} を持つ。両者を型で区別することで、ローカル推論は機外へ何も出さないことをコードを読まずに保証できる。カメラ画像が外部へ送信されないことが、RecognitionActor の効果集合から従う。

3.2 効果推論が実際にバグを検出した

本作業中、ShelfActor.alloc に write_file を書きながら !{fs} を申告し忘れた。処理系は即座にこれを指摘した。

```
$ ./aipl/run_aipl.sh check
[type] method ShelfActor.alloc: effect set incomplete
-- declared {mut, net} but uses {fs}; missing: {fs}
```

これは「効果注釈が飾りではない」ことの実証である。最終的に永続化を SafetyActor へ集約し、現在は次のとおり指摘ゼロである。

```
$ ./aipl/run_aipl.sh check
[type] no issues.
```

3.3 ブラウザ側の推論との一致

シミュレータ画面のアクターカードは、申告を読まずにメソッド本体の primitive から効果を独立に推論して表示する。11 クラス中 9 クラスで推論と申告が完全一致した。残る 2 つ（PlannerActor, Coordinator）は申告のほうが広いが、これは正常である。両者は呼び出し先から伝播する !{ai} を申告しており、処理系は呼び出し連鎖を追うのに対し、ブラウザ側は直接の primitive しか見ないためである。

4 TinyML — 手首カメラによる認識

4.1 構成

手首カメラはグリップ脇に載る実カメラであり、96×96 のレンダターゲットへ描画される。読み出した画素がそのまま TinyML の入力になり、画面右上に表示しているのは推論に渡すのと同じ画素である。

入力	96×96 RGBA → 中央 62% を切出し → 8×8 平均プーリング → 192 次元
構造	192 → 12 (tanh) → 3 (softmax)、 2,355 パラメータ
学習	交差エントロピー + モーメンタム付き SGD。numpy/PyTorch 非依存の純 Python
データ	実レンダ画像 600 枚（3 クラス各 200 枚）。姿勢・撮像高さ・部品の向きを振る

フレームワークに依存しないことが要点である。行列積を素で書いた数十行であり、実機の Raspberry Pi でも Xinu 上でもそのまま動く規模に収めてある。学習・シミュレータ推論・AIPL 推論の 3 者が同一の前処理関数を通る。

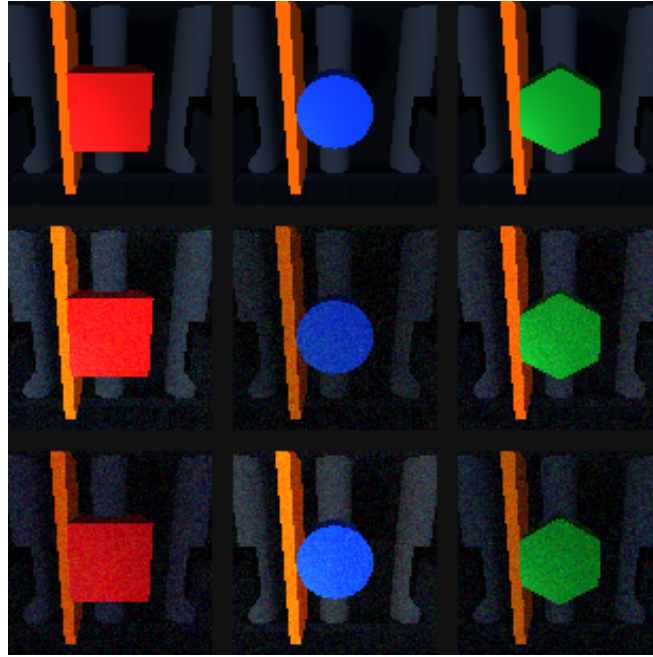


Figure 2: 手首カメラの実画像（左から赤・青・緑）。上段は劣化なし、中下段が実機相当の劣化モデル適用後。露出・色温度・ノイズがフレームごとに揺れる。表示も推論もこの劣化後の画素を使う。

4.2 カメラ劣化モデル

初期の実装ではテスト精度が *epoch 0* で 100% に達した。理想的すぎる画像に対し、分類器が平均色だけで解いてしまい、何の仕事もしていなかったためである。そこで実機の安価な USB カメラ相当の劣化を導入した（図2）。

- 露出ゆらぎ：ゲイン $\sim U(0.55, 1.45)$
- ホワイトバランス偏り：チャンネルごとに $\sim U(0.85, 1.15)$
- センサノイズ： $\sigma = 12/255$ のガウス雑音

4.3 結果 — 正直な評価

劣化モデルを入れてもなお、**テスト精度は 100%**（120 枚、混同行列は完全対角）であった。

正解 \ 予測	赤	青	緑
赤・立方体	41	0	0
青・円柱	0	37	0
緑・六角柱	0	0	42

この結果は「難しい視覚課題を解いた」と読んではいならない。3色は彩度が高く、色相は露出変動でも保たれるため、課題自体が本来易しい。モデルも学習も本物であり、実レンダ画像から学習して実画素で推論しているが、色分けは元来容易なタスクである。なお途中で 65.8% という数値が出たが、これは課題の難しさではなく学習率 0.35 が発散していた筆者のバグであり、0.02 で 100% に戻った。

認識結果のみで棚が決まる点は重要である。部品の真の色は答え合わせにしか使っておらず、誤認識すれば本当に違う色の棚へ運ばれる。したがって後述の「色一致」検証は視覚系の end-to-end 検証になっている。

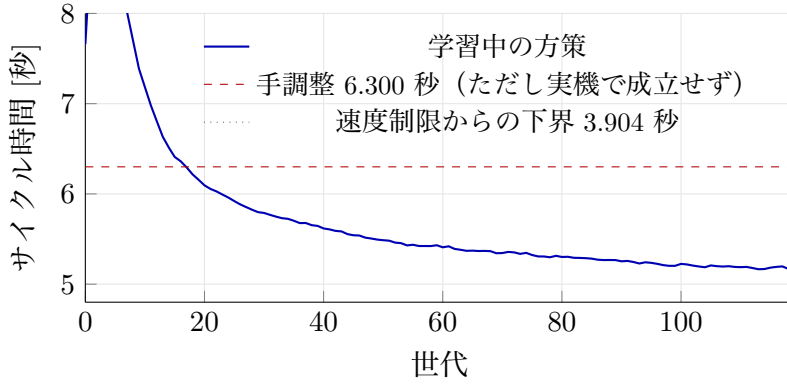


Figure 3: CEM の収束。第 0 世代は制約違反 1.105 を含むため時間が長く見えるが、第 10 世代以降は違反 0 を保ったまま短縮している。

5 強化学習 — サイクル時間の最小化

5.1 定式化

方策 $\phi \in \mathbb{R}^{14}$ は幾何 3 次元と時間 11 次元からなる。

$$\phi = (\underbrace{u_{\text{carry}}, y_{\text{carry}}, h_{\text{hover}}}_{\text{幾何}}, \underbrace{T_1, \dots, T_{11}}_{\text{各フェーズの所要時間}})$$

幾何を変えると全フェーズの関節変位が非線形に変わるため、時間だけを詰める問題ではない。ここが Cross-Entropy Method (CEM) を用いる理由である。報酬は

$$R(\phi) = -\overline{T_{\text{cycle}}}(\phi) - W \cdot \overline{V}(\phi), \quad W = 50$$

であり、 V は制約違反量である。制約は次のとおり。

- ・サーボ角速度上限：min-jerk 補間のピーク速度は平均の 1.875 倍であるから、各関節 j について $1.875 \Delta q_j / T \leq v_{\text{max}}$
- ・可動域：全サーボ指令角が $0-180^\circ$
- ・到達性：全ポーズが逆運動学で解ける
- ・干渉：旋回搬送中に柵および既設部品と当たらない
- ・グリッパ最小開閉時間：0.18 秒

評価は 12 スロット全部を巡る決定論的タスク集合であり、乱数で有利不利が出ない。 W を十分大きく取り、違反が残る解は採用しない（適用スクリプトが拒否する）。これは重要で、罰則が小さいと最適化は「少し制約を破って時間を稼ぐ」解を選ぶ。実際 W が小さい設定では違反 0.141 を残した解が最良となった。

5.2 結果

	手調整	学習後
サイクル時間	6.300 秒	5.164 秒 (−18.0%)
制約違反	3.152	0.000
搬送引込半径 u_{carry}	0.750	0.722
搬送高さ y_{carry}	1.900	1.539
ホバー高さ h_{hover}	0.400	0.239

5.3 最も重要な発見 — ベースラインは実機で動かなかった

表中の「制約違反 3.152」は誤植ではない。手調整のベースラインは、実機のサーボが出せない速度を要求していた。内訳は次のとおりである。

フェーズ	サーボ	要求ピーク角速度	上限比
⑪ホームへ戻る	ID1 / ID5	347.0°/s	1.93 倍
⑥棚へ旋回	ID1 / ID5	290.6°/s	1.61 倍

⑥旋回は ID1 を 155° もの範囲で 1.0 秒で振る指令であり、上限 180°/s の 1.61 倍に相当する。CG 上は何事もなく動くため、画面を見ている限り絶対に気づかない。

強化学習の解は、この 2 フェーズを遅くした（⑥旋回 1.00 → 1.63 秒、⑪ホーム 0.70 → 1.36 秒）うえで、余裕のあったフェーズを大きく詰めている（③下降 0.50 → 0.13 秒、⑧設置 0.50 → 0.14 秒、⑩退避 0.45 → 0.14 秒）。すなわち得られたのは単なる高速化ではなく、実機で成立する動作でありながら、成立しなかった従来より速い動作である。

5.4 学習した方策の実装への反映と実測

学習結果は `rl/apply_policy.py` により `src/layout.js`（3D シミュレータ）と `aipl/dofbot_xinu.abcl`（実機へ出る `ms` 指令）の両方へ書き戻される。書き戻した後、実際にシミュレータを走らせてサイクル時間を実測した。

実測サイクル時間（3D シミュレータ実走行）	5.215 秒
方策の設定値	5.164 秒
差	0.051 秒

短縮は学習側のモデル上だけの話ではなく、実際の描画・逆運動学・状態機械を通した実走行でも再現されている。なお学習解は下界 3.904 秒の 132% であり、まだ余地はある。

6 全体の効率と検証

6.1 1 サイクルの AIPL

`PlannerActor.run_one` が作業の全体である。駆動も永続化も持たないことに注意されたい。

```
method run_one() !{ai, net} {
  var st = now belt.arrived(); // ストップで部品が止まっている

  now claw.write(180, 195); // (1) グリッパを開く
  var qa = now kin.solve(STX, STY + HOVER, STZ);
  now mover.to(qa, 380); // (2) 部品の真上へ(撮像姿勢)

  var frame = now camera.capture(); // 手首カメラで撮像
  var res = now recog.classify(frame); // TinyML 認識 -> ai
  var cls = res[0];
  var slot = now shelf.alloc(cls); // 色の一致する棚の空き枠
  var P = now kin.slot_pose(cls, slot);

  var qg = now kin.solve(STX, STY, STZ);
  now mover.to(qg, 130); // (3) 把持点まで下降
  now claw.write(105, 195); // (4) 把持(部品幅ぶんだけ閉じる)
  ...
  now mover.to(cr, 1631); // (6) 棚の方向へベース旋回
  ...
  send safety.complete(cls, slot); // 永続化は SafetyActor の仕事
  var fd = now belt.feed(); // 次の部品を停止位置へ
  reply(cls);
}
```


このソースは実際に処理系で実行され、実機と同じ API ヘサーボ指令を出す。

```
$ ./aipl/run_aipl.sh
[dofbot] 12 Xinu(6サーボ x 2) + TinyML 起動。
[servo] ID6 -> 180.0deg in 195ms
[servo] ID1 -> 8.0deg in 380ms
[servo] ID2 -> 97.4deg in 380ms
TinyML 認識 -> 緑・六角柱 信頼度 99.9%
[belt ] feed
[dofbot] 6 サイクル完了。サーボ指令 258 回。
```

6.2 検証

主張はすべて機械検査に紐づけてある。

検査	内容	結果
cell.mjs	全 12 スロット + コンベアの到達性、棚どうし・棚とコンベア・搬送経路・グリッパと背板の干渉、サーボ可動域	干渉 0・到達不能 0
verify.mjs	実走行。把持点↔部品中心の距離、搬送中の追従、置いた棚の色一致、設置誤差、全フェーズ実行	全項目合格
aipl_sync.mjs	ブラウザ側と AIPL の字面結合（クラス名・primitive・行対応・効果）	同期
cycle_time.mjs	サイクル時間の実測と設定値の一致	差 0.051 秒
fault.mjs	カメラ障害注入 → 不良品排出 → 復帰	巡回維持
run_aipl.sh check	AIPL の型・効果検査	no issues

主要な実測値は次のとおりである。

把持誤差（指を閉じた瞬間の把持点 ↔ 部品中心）	0.000 mm
搬送中の追従誤差（最大）	0.000 mm
設置誤差（解放時の部品 ↔ スロット中心）	0.000 mm
TinyML 認識（実走行）	6/6 正解
色一致（置いた棚）	全数一致
サーボ指令角	ID1-ID6 すべて 0-180° 内

把持誤差が厳密に 0なのは、逆運動学が解析解であり、把持点が部品中心に一致した瞬間に剛体固定（attach）しているためである。旧実装は関節角がハードコードされており、把持点は $y \approx 4.9$ の空中にあった。部品（ $y \approx 0.9$ ）には物理的に届いておらず、部品をハンドへ補間で吸着させて「掴んでいるように見せて」いた。

7 実機接続の経路

デバイス層はバックエンド差替え式であり、AIPL ソースは 1 行も変えずに実機へ向けられる。

DOFBOT_BACKEND=log	（既定）指令を標準出力へ。実機非接続でも完走する
DOFBOT_BACKEND=armLib	DOFBOT の Raspberry Pi 上で Yahboom Arm_Lib を叩く
DOFBOT_BACKEND=http	DOFBOT_URL のブリッジへ POST（LAN 越し）

8 限界と前提

本報告の主張を読む上で、次の 4 点を明示しておく。

1. 実機は動かしていない。物理の DOFBOT はネットワーク上に存在しない。armLib / http 経路は用意してあるが未検証である。

2. $v_{\max} = 180^\circ/\text{s}$ は仮定であり、データシート値ではない。Yahboom の 15kg バスサーボは無負荷で約 $300^\circ/\text{s}$ とされるが、負荷とアーム慣性を見込んで derate した。§5.3 の「ベースラインは実機で動かない」という結論はこの仮定に依存する。仮定を $300^\circ/\text{s}$ に緩めれば⑥旋回は成立し、⑪ホームのみが違反となる。
3. 視覚課題は易しい。テスト精度 100% は課題の平易さの反映であり、TinyML の能力の証明ではない。
4. 強化学習は解析的下界に届いていない。学習解は下界の 132% である。また力学モデルは min-jerk のピーク速度による近似であり、トルク・慣性・バックラッシュを含まない。

9 再現手順

```
# 3D シミュレータ
./run.sh # localhost:8022

# AIPL (本物の処理系で実行)
./aipl/run_aipl.sh check # 型 + 効果検査
./aipl/run_aipl.sh # 実行 (サーボ指令を出力)

# TinyML
node rl/capture_dataset.mjs # 手首カメラの実画像から学習データ
python3 rl/train_tinyml.py # 学習 -> aipl/tinyml_model.json

# 強化学習
python3 rl/dofbot_rl.py # CEM -> rl/rl_policy.json
python3 rl/apply_policy.py # 方策を layout.js と .abcl へ書き戻す
python3 rl/apply_policy.py --restore # 手調整へ戻す

# 検証 (方策を書き戻したら必ず通す)
node test/cell.mjs # 幾何: 到達性・干渉
node test/verify.mjs # 実走行: 把持誤差・色一致
node test/aipl_sync.mjs # ブラウザ側と AIPL のずれ
node test/cycle_time.mjs # サイクル時間の実測
```

10 結び

本作業で得られたもののうち、価値の順に 3 点を挙げる。

第一に、効果推論は実際に働いた。ShelfActor.alloc の申告漏れは人間のレビューではなく処理系が見つけた。効果を型に載せる主張は、それが実装バグを捕まえたという一点で裏付けられる。

第二に、強化学習は「速くする」以上のことをした。副産物として、手調整の動作が実機のサーボ速度を超えていたことを暴いた。これは CG を眺めていても、コードを読んでも見つからない種類の誤りである。制約を明示的に書き下し、それを破る解を採用しないと決めたことで初めて表に出た。

第三に、TinyML は本物になったが、課題は易しかった。手首カメラは実際に描画され、分類器は実画素から学習され、その出力だけで柵が決まる。しかし精度 100% は課題の平易さを反映したものであり、そう読まれるべきである。

共通するのは、見た目では分からないものを機械に検査させると、実装は必ず嘘をついているということである。柵は重なり、サーボは可動域に張り付き、グリッパは背板を突き抜け、カメラは自分の筐体の内側を写し、効果推論のパネルは例外も出さずに死んでいた。いずれも画面上は正常に見えていた。